KTM Mall Ad-Space Booking System

Full Project Documentation

Version 1.0 | May 2026 | Dreams Media & Marketing

Stack: n8n + Evolution API + Gemini 2.0 Flash Lite + Google Sheets + Static HTML

Version: 1.0 | **Date:** May 2026 | **Author:** Dreams Media & Marketing

Stack: n8n + Evolution API + Gemini 2.0 Flash Lite + Google Sheets + Static HTML

TABLE OF CONTENTS

1. [Project Overview](#1-project-overview)
2. [System Architecture](#2-system-architecture)
3. [Infrastructure — What's Running Where](#3-infrastructure)
4. [File Structure](#4-file-structure)
5. [Quick Start Manual — Run This Project](#5-quick-start-manual)
6. [Workflow Deep Dives](#6-workflow-deep-dives)
7. [Google Sheet Structure](#7-google-sheet-structure)
8. [All Credentials & Keys](#8-all-credentials--keys)
9. [Errors Encountered & How They Were Solved](#9-errors-encountered--solutions)
10. [What Took Too Long & Why](#10-what-took-too-long--why)
11. [LLM Build Guide — Reproduce This Project From Scratch](#11-llm-build-guide)
12. [Testing Checklist](#12-testing-checklist)

---

1. PROJECT OVERVIEW

Client: The Kathmandu Mall, Sundhara, Kathmandu

Partner: Dreams Media & Marketing (exclusive advertising partner)

Contact: +977-9860499000 | info.dreamsmarketing@gmail.com | hoardingboardnepal.com

What This System Does

A lead generation and WhatsApp AI sales bot for selling advertising spaces inside The Kathmandu Mall.

Full flow:

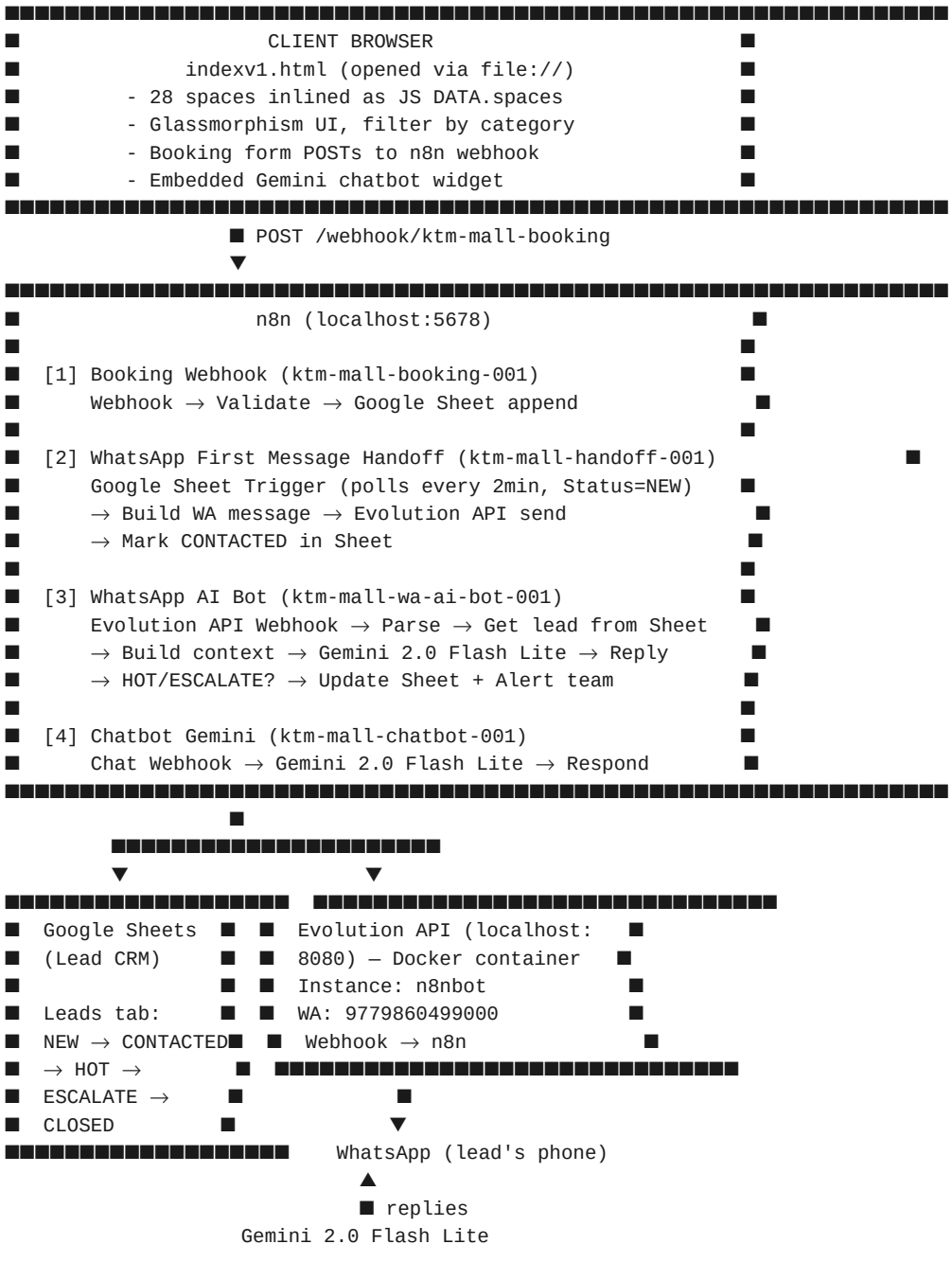
```
Visitor opens indexv1.html (file://)
  → Browses 28 ad spaces with photos, prices, filters
  → Fills booking enquiry form
```

```
  → n8n saves lead to Google Sheet (Status: NEW)
  → n8n sends first WhatsApp to lead via Evolution API (Status: CONTACTED)
→ Lead replies on WhatsApp
  → Gemini 2.0 Flash Lite responds as "Sales AI Agent"
  → Buying signal detected? → Mark HOT/ESCALATE + alert team
```

Ad Inventory

- 28 advertising spaces across: Façade/Exterior, Main Gate, Parking, Pillars, Stairs & Escalator, Floor Hoardings, Interior Spots, Lift
- Price range: Rs. 4,000/month → Rs. 25,00,000/year

2. SYSTEM ARCHITECTURE



```
                    (Sales AI Agent persona)
```

3. INFRASTRUCTURE

Machine

- OS: Windows 10/11
- n8n: npm global install, runs as process
- Evolution API: Docker container

Services & Ports

Service	URL	Notes
n8n	http://localhost:5678	Start: `n8n start`
Evolution API	http://localhost:8080	Docker container: `evolution_api`
Evolution Manager UI	http://localhost:8080/manager	Login with API key
PostgreSQL (Evo DB)	localhost:5432	Docker container: `evolution_postgres`

Docker Containers

```
# Check status
docker ps

# Containers:
# evolution_api      — evoapicloud/evolution-api:v2.3.1
# evolution_postgres — postgres:15
```

Starting Everything

```
# 1. Start Docker containers (if not running)
docker start evolution_api evolution_postgres

# 2. Start n8n (run in background)
n8n start

# 3. Verify
curl http://localhost:5678/healthz     # → {"status":"ok"}
curl http://localhost:8080/            # → Evolution API response
```

4. FILE STRUCTURE

```
C:\Users\gyanendra karki\Desktop\ktm mall\
■■■ indexv1.html                ← THE WEBSITE (single file, open from file://)
■■■ data\
■   ■■■ spaces.json             ← Reference copy of 28 spaces (NOT loaded by site)
■■■ assets\
■   ■■■ img\spaces\             ← 134 web-optimized JPEGs (max 1400px, q82)
■■■ n8n\
■   ■■■ booking-webhook.json    ← Workflow 1: Lead capture → Google Sheet
```

```
■   ■■■ chatbot-gemini.json       ← Workflow 4: Website chatbot
■   ■■■ openclaw-handoff.json     ← Workflow 2: Sheet trigger → first WA message
■   ■■■ whatsapp-ai-bot.json      ← Workflow 3: WA replies → Gemini → auto-reply
■■■ extracted\                    ← Raw docx output (original photos in word\media\)
■■■ PROJECT_DOCUMENTATION.md      ← This file
```

---

5. QUICK START MANUAL

EVERY TIME YOU START THE MACHINE

Step 1 — Start Docker (Evolution API)

```
docker start evolution_api evolution_postgres
```

Wait 10 seconds, then verify:

```
curl http://localhost:8080/instance/fetchInstances?instanceName=n8nbot \
  -H "apikey: B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE"
```

Should return `"connectionStatus":"open"`. If it says `"close"`:

```
curl -X POST http://localhost:8080/instance/restart/n8nbot \
  -H "apikey: B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE"
```

Step 2 — Start n8n

```
n8n start
```

Wait ~15 seconds. Visit http://localhost:5678 to confirm it's running.

Step 3 — Open the website

Open `C:\Users\gyanendra karki\Desktop\ktm mall\indexv1.html` in browser directly (double-click or drag to Chrome). It runs from `file://` — no server needed.

That's it. The system is live.

IF WHATSAPP DISCONNECTS

1. Go to http://localhost:8080/manager
2. Login with API key: `B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE`
3. Click your instance → **Get QR Code**
4. Open WhatsApp on phone → **Linked Devices** → **Link a Device** → scan QR
5. Scan within 20 seconds — QR expires fast

If dashboard is slow to load: The dashboard can be slow when the WhatsApp account has many chats (4000+ messages). This is cosmetic only — the API still works. Just wait or ignore the spinner. The bot works regardless.

If dashboard shows "Application taking too long":

```
# Restart the instance via API (don't need the UI)
curl -X POST http://localhost:8080/instance/restart/n8nbot \
  -H "apikey: B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE"
```

IF n8n WORKFLOWS ARE INACTIVE

Go to http://localhost:5678 → Workflows → make sure all 4 KTM Mall workflows show green (Active).

If any is inactive, click it → toggle Active ON → Save.

If WhatsApp First Message Handoff fails to activate (error: "Node does not have any credentials set"):

1. Open the workflow
2. Click **"New Lead in Sheet"** node → assign Google Sheets credential
3. Click **"Mark CONTACTED in Sheet"** node → assign same credential
4. Save → Activate

REIMPORTING WORKFLOWS (if n8n DB is reset)

```
cd "C:\Users\gyanendra karki\Desktop\ktm mall\n8n"

n8n import:workflow --input=booking-webhook.json
n8n import:workflow --input=openclaw-handoff.json
n8n import:workflow --input=whatsapp-ai-bot.json
n8n import:workflow --input=chatbot-gemini.json

n8n update:workflow --id=ktm-mall-booking-001 --active=true
n8n update:workflow --id=ktm-mall-handoff-001 --active=true
n8n update:workflow --id=ktm-mall-wa-ai-bot-001 --active=true
n8n update:workflow --id=ktm-mall-chatbot-001 --active=true
```

Then restart n8n and **manually assign Google Sheets credentials** in the UI (see above).

TEST THE FULL PIPELINE

Test 1 — Booking form saves to sheet:

```
curl -X POST http://localhost:5678/webhook/ktm-mall-booking \
  -H "Content-Type: application/json" \
  -d '{"spaceId":"TEST-01","space":"Main Gate Hoarding","price":"Rs. 60000","name":"Test Lead","phone":"9800000001
```

→ Check Google Sheet for new row with Status=NEW

Test 2 — WhatsApp bot replies:

```
curl -X POST http://localhost:5678/webhook/ktm-mall-wa-bot \
  -H "Content-Type: application/json" \
  -d '{"event":"messages.upsert","instance":"n8nbot","data":{"key":{"remoteJid":"977XXXXXXXXXX@s.whatsapp.net","fr
```

→ Check if Gemini reply is sent back to that WhatsApp number

Test 3 — Full flow:

1. Open indexv1.html → fill booking form with a real WhatsApp number
2. Submit → check Google Sheet (row should appear, Status=NEW)
3. Wait 2 minutes → check if WhatsApp gets the first message (from handoff workflow)
4. Reply to that WhatsApp → bot should respond with Gemini AI reply

---

6. WORKFLOW DEEP DIVES

Workflow 1: Booking Webhook (`ktm-mall-booking-001`)

Trigger: POST to `http://localhost:5678/webhook/ktm-mall-booking`

Called by: indexv1.html booking form submit

Nodes:

1. `Booking Webhook` — receives POST, returns 200
2. `Validate & Sanitize` — Code node: cleans phone number, adds timestamp, generates space_id
3. `If Valid` — checks required fields exist
4. `Append to Google Sheet` — appends to `Leads` tab, Status=NEW
5. `Respond OK` / `Respond Error` — returns JSON response to website

Key data flow:

```
// Input from form:
{ spaceId, space, price, name, phone, whatsapp, company, source }

// After Validate & Sanitize adds:
{ ...above, ts_local, status: "NEW", whatsapp_url: "https://wa.me/977XXXXXXX" }
```

Workflow 2: WhatsApp First Message Handoff (`ktm-mall-handoff-001`)

Trigger: Google Sheets polling every 2 minutes — watches for new rows

Purpose: Sends the first WhatsApp message to every new lead

Nodes:

1. `New Lead in Sheet` — GoogleSheetsTrigger polls every 2 min
2. `Only NEW with Phone` — Filter: Status=NEW AND WhatsApp is not empty
3. `Build WA Message` — Code node: crafts personalized first message
4. `Send via Evolution API` — HTTP POST to Evolution API sendText
5. `Mark CONTACTED in Sheet` — Updates Status → CONTACTED
6. `Notify Sales Team` — HTTP POST to Evolution API (alerts team number)
7. `Log Error` — Code node for error logging

Evolution API call:

```
POST http://localhost:8080/message/sendText/n8nbot
Headers: { "apikey": "B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE" }
Body: {
  "number": "977XXXXXXXXX",
  "text": "Hello [Name]! ..."
}
```

Workflow 3: WhatsApp AI Bot (`ktm-mall-wa-ai-bot-001`)

Trigger: POST to `http://localhost:5678/webhook/ktm-mall-wa-bot`

Called by: Evolution API webhook (every incoming WhatsApp message)

Nodes:

1. `Evolution API Webhook` — receives all WA events
2. `Parse Incoming Message` — Code: extracts sender JID, message text, phone number

3. `Skip non-messages` — Filter: only process actual text messages (not status updates, delivery receipts)
4. `Get Lead from Sheet` — Looks up sender's phone in Leads tab
5. `Build AI Context` — Code: assembles system prompt + conversation context
6. `Gemini 1.5 Flash` — HTTP POST to Gemini 2.0 Flash Lite API
7. `Parse AI Reply + Detect Signal` — Code: extracts reply text, detects [HOT_LEAD] / [ESCALATE] tags
8. `Reply to Lead on WhatsApp` — Evolution API sendText back to lead
9. `Hot Lead or Escalate?` — IF node: checks for buying signal
10. `Update Sheet: HOT or ESCALATE` — Updates Status in Google Sheet
11. `Alert Sales Team on WhatsApp` — Sends alert to team number (9779860499000)
12. `Respond 200 to Evolution` — Returns 200 OK to Evolution API

AI Persona (system prompt summary):

• Name: **Sales AI Agent**
• Company: Dreams Media & Marketing
• Role: Sales executive for KTM Mall ad spaces
• Language: Replies in whatever language client uses (Nepali or English)
• Reply length: 2–4 sentences max
• Buying signal tags: `[HOT_LEAD]` or `[ESCALATE]` appended to reply

Gemini API call:

```
POST https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash-lite:generateContent?key=GEMINI_API_
```

Workflow 4: Chatbot Gemini (`ktm-mall-chatbot-001`)

Trigger: POST to `http://localhost:5678/webhook/ktm-mall-chatbot`

Called by: Embedded chat widget in indexv1.html

Nodes:

1. `Chat Webhook` — receives chat message from website
2. `Build Gemini Request` — Code: builds Gemini API payload with system prompt
3. `Gemini 1.5 Flash` — HTTP POST to Gemini 2.0 Flash Lite API
4. `Parse Reply` — Code: extracts text from Gemini response
5. `Respond` — Returns reply to website widget

---

7. GOOGLE SHEET STRUCTURE

Sheet ID: `1HxyzZNhmPXU3LLjk1STczPQFA4XMktWsruQ6MmKXHJc`

Tab name: `Leads`

Column	Description	Example
Timestamp	Local Nepal time	2026-05-06 09:15:00
SpaceID	Internal space code	MAIN-GATE-001
Space	Human readable name	Main Gate Hoarding (Large)

Column	Description	Example
Price	Price string	Rs. 60,000/month
Name	Lead full name	Ram Sharma
Phone	Phone number	9800000001
WhatsApp	wa.me link	https://wa.me/9779800000001
Company	Company name	ABC Pvt Ltd
Source	Traffic source	website
Status	Lead status	NEW / CONTACTED / HOT / ESCALATE / CLOSED
Owner	Assigned sales rep	(manual entry)
Notes	AI notes / timestamps	HOT LEAD detected at 2026-05-06...

Status flow:

```
NEW → CONTACTED → HOT → ESCALATE → CLOSED
 ↑         ↑         ↑         ↑
 Form    Handoff    Gemini  Gemini/Human
submit  workflow   signal  override
```

---

8. ALL CREDENTIALS & KEYS

Item	Value
Evolution API Key	`B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE`
Evolution Instance	`n8nbot`
WhatsApp Number	`9779860499000`
Google Sheet ID	`1HxyzZNhmPXU3LLjk1STczPQFA4XMktWsruQ6MmKXHJc`
Gemini API Key	`AIzaSyCalNkYiHnqc5E-l_a6MdZEmyGsLl6X5fA`
Team Alert JID	`9779860499000@s.whatsapp.net`
n8n URL	`http://localhost:5678`
n8n Encryption Key	`vAH2UQ6lzJCz7roIQEVKPzwT1cZYMxPQ`
Evolution API Docker image	`evoapicloud/evolution-api:v2.3.1`
docker-compose location	`C:\Users\gyanendra karki\Desktop\docker\EvolutionAPI\docker-compose.yml`

Google OAuth Client (for Google Sheets OAuth in n8n):

- Client ID: `206087599148-vvvr8dd9joqhgpj38qt8ql31ld64trad.apps.googleusercontent.com`
- Owner account: `sanamthapakarki@gmail.com`
- Business Gmail: `info.dreamsmarketing@gmail.com`

---

9. ERRORS ENCOUNTERED & SOLUTIONS

ERROR 1: Evolution API QR never generates (noise handshake failure)

Symptom: QR code spinner never appears. Logs show "Connection Failure" loop.

Root cause: `atendai/evolution-api:latest` ships Baileys `2.3000.1015901307` which WhatsApp has blocked.

Solution: Switch Docker image to `evoapicloud/evolution-api:v2.3.1` which ships Baileys `2.3000.1038807311`.

```
# docker-compose.yml — change image line:
image: evoapicloud/evolution-api:v2.3.1  # NOT atendai/evolution-api:latest
```

Time lost: ~2 hours debugging wrong Baileys version.

ERROR 2: Evolution API dashboard "Application taking too long to load"

Symptom: Dashboard spins forever after QR scan.

Root cause: Two causes:

1. After QR scan, a `fetchProps` init query to WhatsApp servers times out (408 error), leaving instance in broken state
2. Large chat history (4000+ messages) causes dashboard UI to time out rendering

Solution:

```
# Restart the instance via API (not the Docker container)
curl -X POST http://localhost:8080/instance/restart/n8nbot \
  -H "apikey: B7kR9mX2pQ4nL6wT8vY3cF5hJ0dA3sE"
```

For large history: the dashboard is cosmetic — bot works even if dashboard is slow. Use the API directly to check status.

Key insight: Always verify WhatsApp connection via API, not the UI.

ERROR 3: n8n import fails — "NOT NULL constraint failed: workflow_entity.id"

Symptom: `n8n import:workflow` throws SQLite constraint error.

Root cause: Workflow JSON missing the `"id"` field.

Solution: Add `"id"` field to workflow JSON before import:

```
import json
d = json.load(open('workflow.json', encoding='utf-8'))
wf = d if not isinstance(d, list) else d[0]
wf['id'] = 'my-workflow-id-001'
wf['tags'] = []
json.dump(d, open('workflow.json', 'w', encoding='utf-8'), ensure_ascii=False, indent=2)
```

Rule: Every workflow JSON imported into n8n MUST have `"id"` (string) and `"tags": []`.

ERROR 4: n8n import fails — "NOT NULL constraint failed: workflows_tags.tagId"

Symptom: Import fails even after adding `id` field.

Root cause: `tags` array contains objects with null IDs (e.g. `[{"id": null, "name": "foo"}]`).

Solution: Set `"tags": []` (empty array, not array with null objects).

ERROR 5: booking-webhook.json — Google Sheet documentId broken

Symptom: After text replacement of `GOOGLE_SHEET_ID`, the field became `$env.1DiEOFO...`

Root cause: The original workflow used `{{ $env.GOOGLE_SHEET_ID }}` expression. String replacing `GOOGLE_SHEET_ID` with the actual ID broke the expression to `{{ $env.1DiEOFO... }}`.

Solution: Use regex to replace the entire expression, not just the variable name:

```
import re
txt = re.sub(r'=\{\{[^}]*GOOGLE_SHEET_ID[^}]*\}\}', SHEET_ID, txt)
```

Rule: When replacing n8n expression variables, replace the whole `={{ $env.VAR }}` expression, not just the variable name.

ERROR 6: WhatsApp First Message Handoff fails to activate — "Node does not have any credentials set"

Symptom: `ktm-mall-handoff-001` retries activation forever with credential error.

Root cause (deep): n8n v2.12 uses a versioning system with three tables: `workflow_entity` (draft nodes), `workflow_history` (all versions), `workflow_published_version` (which version is live). Direct DB edits to `workflow_entity.nodes` do NOT affect what n8n loads for trigger activation — it loads from `workflow_history` via `activeVersionId`. Additionally, n8n API requires the `activeVersion` sub-object for node inspection.

What was tried:

- Patching `workflow_entity.nodes` → did not work (n8n loads from history)
- Patching `workflow_history` → patch applied but didn't survive because version lookup was wrong
- Direct DB credential injection → complicated by n8n's encrypted credential store

Actual fix (manual — required for Google OAuth):

Go to n8n UI → open workflow → click each Google Sheets node → assign credential from dropdown → Save.

This is the ONLY reliable method for Google OAuth credentials because n8n must validate the OAuth token at activation time.

Key insight: Google Sheets TRIGGER nodes (`googleSheetsTrigger`) are stricter than regular nodes — n8n actually calls the Google API during activation to register the poll. If the OAuth token is expired or not properly linked, activation fails even if the credential ID is in the node JSON.

Time lost: ~3 hours trying to bypass the manual step programmatically.

ERROR 7: n8n API key returns 401 Unauthorized

Symptom: `curl http://localhost:5678/api/v1/workflows` returns `{"message":"unauthorized"}`.

Root cause: The API key JWT had an `exp` field set to a past date (expired).

Solution: Use the API key without expiry (the "CALL AUTOMATION" key in `user_api_keys` table has no `exp` in the JWT payload).

Prevention: When creating n8n API keys, set "Never Expire" or use long expiry dates. Check DB:

```
import sqlite3
db = sqlite3.connect(r'path\to\database.sqlite')
rows = db.execute('SELECT label, apiKey FROM user_api_keys').fetchall()
```

```
# Decode JWT payload (base64 middle part) to check exp field
```

ERROR 8: Gemini model name mismatch

Symptom: Workflows built with `gemini-1.5-flash` needed to be upgraded.

Solution: Simple string replace in workflow JSON + re-import:

```
txt = txt.replace('gemini-1.5-flash:generateContent', 'gemini-2.0-flash-lite:generateContent')
```

Note: `gemini-2.0-flash-lite` is the current cheapest/fastest model. Check https://ai.google.dev/gemini-api/docs/models for current model IDs.

10. WHAT TOOK TOO LONG & WHY

Issue	Time Lost	Root Cause	Prevention
Wrong Baileys version blocking QR	~2 hrs	Used `atendai/evolution-api:latest` which ships blocked Baileys	Always use `evoapicloud/evolution-api:v2.3.1` or check Baileys version before deploying
Dashboard loading issue misleading debugging	~30 min	Confused cosmetic dashboard slowness with actual connection failure	Always check WA connection via API (`/instance/fetchInstances`), not UI
n8n credential injection in DB	~3 hrs	n8n's versioning system (workflow_entity vs workflow_history vs workflow_published_version) is not obvious. Google OAuth trigger validation can't be bypassed	For Google OAuth workflows: ALWAYS set credentials manually in n8n UI. Don't try to inject via DB.
Broken `$env.` expression after string replace	~20 min	Naive text replacement broke n8n expression syntax	Use regex to replace full `={{ $env.VAR }}` expressions
Workflow import errors (missing id, bad tags)	~30 min	n8n requires specific JSON structure for imports	Template: every workflow JSON needs `"id"` string + `"tags": []`

11. LLM BUILD GUIDE

For any LLM building a similar WhatsApp AI sales bot with n8n + Evolution API + Gemini:

PHASE 1: Infrastructure Setup

```
# 1. Run Evolution API with correct image
docker run -d \
  --name evolution_api \
  -p 8080:8080 \
  -e AUTHENTICATION_API_KEY=YOUR_API_KEY \
  evoapicloud/evolution-api:v2.3.1
```

```
# CRITICAL: Use evoapicloud/evolution-api:v2.3.1 NOT atendai/evolution-api:latest
# Reason: atendai latest ships blocked Baileys version
```

PHASE 2: Evolution API Instance Setup

```
# Create instance
curl -X POST http://localhost:8080/instance/create \
  -H "apikey: YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"instanceName":"n8nbot","integration":"WHATSAPP-BAILEYS"}'

# Register webhook (MUST point to n8n BEFORE scanning QR)
curl -X POST http://localhost:8080/webhook/set/n8nbot \
  -H "apikey: YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "http://localhost:5678/webhook/ktm-mall-wa-bot",
    "webhook_by_events": false,
    "webhook_base64": false,
    "events": ["MESSAGES_UPSERT"]
  }'
```

PHASE 3: Google Sheet Setup

Create sheet with columns:

```
Timestamp | SpaceID | Space | Price | Name | Phone | WhatsApp | Company | Source | Status | Owner | Notes
```

Note the Sheet ID from URL: `docs.google.com/spreadsheets/d/SHEET_ID/edit`

PHASE 4: n8n Workflow Architecture

RULE: Never use `$env.VAR_NAME` in workflow nodes if you want to inject values programmatically. Hardcode values or use n8n Variables.

Workflow JSON template:

```
{
  "id": "unique-workflow-id",
  "name": "Workflow Name",
  "nodes": [...],
  "connections": {...},
  "settings": {},
  "tags": []
}
```

Google Sheets node credential format:

```
{
  "credentials": {
    "googleSheetsOAuth2Api": {
      "id": "CREDENTIAL_ID_FROM_N8N",
      "name": "Google Sheets account"
    }
  },
  "parameters": {
    "documentId": {
      "__rl": true,
      "value": "SHEET_ID",
      "mode": "id"
    },
    "sheetName": {
      "__rl": true,
```

```
      "value": "Leads",
      "mode": "name"
    }
  }
}
```

Evolution API HTTP Request node (send WhatsApp message):

```
{
  "method": "POST",
  "url": "http://localhost:8080/message/sendText/n8nbot",
  "headers": { "apikey": "YOUR_EVOLUTION_API_KEY" },
  "body": {
    "number": "={{ $json.phone }}",
    "text": "={{ $json.message }}"
  }
}
```

Gemini HTTP Request node:

```
{
  "method": "POST",
  "url": "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash-lite:generateContent?key=GEMINI
  "body": {
    "systemInstruction": { "parts": [{ "text": "SYSTEM_PROMPT" }] },
    "contents": [{ "role": "user", "parts": [{ "text": "={{ $json.message }}" }] }],
    "generationConfig": { "temperature": 0.7, "maxOutputTokens": 512 }
  }
}
```

PHASE 5: Import & Activate Workflows

```
# Import (ensure each JSON has "id" and "tags": [])
n8n import:workflow --input=workflow.json

# Activate
n8n update:workflow --id=WORKFLOW_ID --active=true

# Restart n8n for changes to take effect
# Kill node process then: n8n start
```

PHASE 6: Credential Assignment (MANUAL — can't be automated)

Google OAuth credentials MUST be assigned manually in n8n UI:

1. http://localhost:5678 → Workflows → open workflow
2. Click each Google Sheets node → Credential dropdown → select credential
3. Save → Activate

Why it can't be automated: n8n validates the OAuth token by calling Google API during trigger activation. DB injection bypasses this check and causes the trigger to fail at runtime.

PHASE 7: Test Each Layer

```
# Layer 1: Booking webhook
curl -X POST http://localhost:5678/webhook/ktm-mall-booking \
  -H "Content-Type: application/json" \
  -d '{"name":"Test","phone":"9800000001","whatsapp":"9800000001",...}'

# Layer 2: Check Google Sheet for new row
```

```
# Layer 3: Simulate WhatsApp message to bot
curl -X POST http://localhost:5678/webhook/ktm-mall-wa-bot \
  -H "Content-Type: application/json" \
  -d '{"event":"messages.upsert","instance":"n8nbot","data":{...}}'

# Layer 4: Check n8n execution log for errors
# C:\Users\gyanendra karki\Desktop\n8n.log
```

CRITICAL RULES FOR FUTURE BUILDS

1. **Evolution API image:** Always `evoapicloud/evolution-api:v2.3.1` — never `atendai/latest`
2. **Register webhook BEFORE scanning QR** — otherwise first messages are missed
3. **Workflow JSON must have** `"id"` **and** `"tags": []` — or import fails
4. **Google OAuth = manual UI step** — can't be injected via DB
5. **Check WA connection via API not UI** — UI is slow with large history
6. **QR code expires in ~20 seconds** — have phone ready before clicking "Get QR"
7. **n8n restart required after import** — changes don't apply to running process
8. **Gemini model IDs change** — always verify current model at ai.google.dev
9. **Phone numbers for Evolution API** — format: `977XXXXXXXXX` (country code + number, no +)
10. **Google Sheet lookup in bot** — match on `WhatsApp` column using `contains` not `equals` (URL format vs number format)

---

12. TESTING CHECKLIST

Run these in order after any restart or deployment:

- [ ] `curl http://localhost:5678/healthz` → `{"status":"ok"}`
- [ ] `docker ps` shows `evolution_api` and `evolution_postgres` running
- [ ] Evolution API instance status is `"open"` (check via API)
- [ ] All 4 KTM Mall workflows are Active in n8n UI
- [ ] Booking webhook test → row appears in Google Sheet
- [ ] WhatsApp bot test → Gemini reply is sent
- [ ] Hot lead test → Status updates to HOT, team alert sent
- [ ] Website chatbot responds in indexv1.html

---

13. SESSION 2 ERRORS & FIXES (May 2026)

These errors occurred during the activation and Google Sheets integration phase, after all 4 workflows were imported.

ERROR 9: "Sheet with name Leads not found" — Handoff workflow fails to activate

Symptom: Activating `ktm-mall-handoff-001` throws: `There was a problem activating the workflow: 'Sheet with name Leads not found'`

Root cause: The new Google Sheet (`1HxyzZNhmPXU3LLjk1STczPQFA4XMktWsruQ6MmKXHJc`) was freshly created and only had the default tab named `Sheet1`. All 3 workflows (handoff, booking, wa-ai-bot) look for a tab named `Leads` by name (not by ID).

Solution: Rename the `Sheet1` tab to `Leads` directly in Google Sheets:

1. Open the Google Sheet
2. Double-click the `Sheet1` tab at the bottom
3. Type `Leads` → press Enter
4. Also add the header row in row 1:

```
Timestamp | SpaceID | Space | Price | Name | Phone | WhatsApp | Company | Source | Status | Owner | Notes
```

Key insight: Google Sheets workflows in n8n reference sheet tabs by **name**, not by gid. The tab must exist with the exact name before the workflow can activate. Create the `Leads` tab and its header row before activating any workflow.

Time lost: ~15 min.

ERROR 10: Google Sheets "Tables" panel blocks UI interaction

Symptom: Right-clicking the sheet tab to rename it opened a "Tables" side panel instead of the context menu. The panel could not be closed by clicking the X button. All subsequent right-clicks and double-clicks on the tab were intercepted.

Root cause: Google Sheets recently added a "Tables" feature. Double-clicking on a cell in an empty sheet triggers the Tables panel instead of entering edit mode on that cell.

Solution: Use the sheet's built-in **Rename** textbox in the toolbar (accessible via `find` on the page), or use the Name Box at the top-left to navigate, then double-click the tab directly via its element reference.

Workaround used: Located the `Sheet1` tab element by reference (`ref_154`) and double-clicked it to enter inline rename mode, then typed the new name.

ERROR 11: File rename vs sheet tab rename — different operations

Symptom: After using the Rename textbox in the menu bar, the browser tab title changed to "Leads - Google Sheets" but the sheet tab at the bottom still showed `Sheet1`.

Root cause: Google Sheets has two separate "name" concepts:

- **File name** (shown in browser tab title and Google Drive) — changed via the title field top-left
- **Sheet tab name** (the tab at the bottom of the spreadsheet) — changed by double-clicking the tab

Solution: Both need to be renamed separately. For n8n workflows, only the **sheet tab name** matters — the file name is irrelevant.

Key insight: In n8n's Google Sheets node, `"sheetName": {"__rl": true, "value": "Leads", "mode": "name"}` refers to the **tab name**, not the file name.

ERROR 12: Duplicate header columns in Google Sheet

Symptom: When manually adding the header row, columns `Name` and `Phone` were accidentally duplicated (entered twice).

Root cause: Manual data entry error.

Solution: Delete the duplicate columns so the final header row is exactly:

```
A=Timestamp | B=SpaceID | C=Space | D=Price | E=Name | F=Phone | G=WhatsApp | H=Company | I=Source | J=Status | K=
```

Key insight: n8n's Google Sheets append node writes columns by name match. Duplicate column names cause it to write to the first matching column only — data in the second duplicate is silently lost.

ERROR 13: googleSheetsTriggerOAuth2Api requires separate credential from googleSheetsOAuth2Api

Symptom: The Google Sheets Trigger node (used in the Handoff workflow) kept failing with `"Node does not have any credentials set"` even after assigning the standard Google Sheets OAuth2 credential.

Root cause: n8n treats Google Sheets regular nodes and Google Sheets Trigger nodes as two **separate credential types**:

- Regular node: `googleSheetsOAuth2Api`
- Trigger node: `googleSheetsTriggerOAuth2Api`

Both must be independently authorized. Even if you've already authorized `googleSheetsOAuth2Api`, the trigger node requires its own separate OAuth consent flow for `googleSheetsTriggerOAuth2Api`.

Solution:

1. Go to n8n → Credentials → New Credential → **Google Sheets Trigger OAuth2 API**
2. Click "Sign in with Google" → authorize with `info.dreamsmarketing@gmail.com`
3. Assign this new credential to the Google Sheets Trigger node in the Handoff workflow
4. Save → Activate

Key insight: Always check the **exact credential type** required by each node. In the node's credential dropdown, the type name is shown — `googleSheetsTriggerOAuth2Api` vs `googleSheetsOAuth2Api` are different and not interchangeable.

ERROR 14: n8n node typeVersion mismatch causing "not iterable" runtime error

Symptom: Handoff workflow activated but executions failed with `"propertyValues[itemName] is not iterable"` on the Filter node.

Root cause: The workflow JSON had Filter node at `typeVersion: 1`, but the installed n8n version ships Filter at `typeVersion: 2`. The conditions format changed between versions — v1 uses a different schema than v2.

Additionally, other nodes had version mismatches:

- `httpRequest`: needed v4
- `googleSheets`: needed v4
- `googleSheetsTrigger`: needed v1 (NOT v4 — trigger is at v1 in n8n v2.12)
- `code`: needed v2

Solution: Read the installed node versions from n8n's node registry and patch the workflow JSON to match:

```
# Fix node typeVersions before import
node_versions = {
    'n8n-nodes-base.filter': 2,
    'n8n-nodes-base.httpRequest': 4,
    'n8n-nodes-base.googleSheets': 4,
    'n8n-nodes-base.googleSheetsTrigger': 1,
    'n8n-nodes-base.code': 2,
}
```

```
for node in workflow['nodes']:
    t = node.get('type')
    if t in node_versions:
        node['typeVersion'] = node_versions[t]
```

Key insight: typeVersion MUST exactly match what the installed n8n instance supports. A mismatch doesn't always produce an error at import time — it fails silently at runtime with confusing errors like "not iterable".

ERROR 15: OAuth "Forbidden" after copying token between credential types

Symptom: After manually creating `googleSheetsTriggerOAuth2Api` credential in the DB by copying the OAuth token from the existing `googleSheetsOAuth2Api` credential, activation failed with `"Forbidden - perhaps check your credentials?"`.

Root cause: OAuth2 access tokens and refresh tokens are **scope-specific**. The token obtained for `googleSheetsOAuth2Api` may have been issued with different OAuth scopes than required by the trigger node. Additionally, n8n may use different OAuth client configurations internally for each credential type.

Solution: Never copy tokens between credential types. Always do a **fresh OAuth consent flow** for each credential type:

1. Create new credential of the required type in n8n UI
2. Click "Sign in with Google" → complete OAuth flow
3. Google issues a new token with the correct scopes for that credential type

Key insight: OAuth tokens are not portable between n8n credential types even for the same Google account and same API (Google Sheets). Always authorize fresh.

---

14. FINAL SYSTEM STATUS (May 2026)

Component	Status
Evolution API (n8nbot)	Connected — WhatsApp 9779860499000
n8n	Running — localhost:5678
Workflow: KTM Mall Booking Webhook	Active (green)
Workflow: KTM Mall WhatsApp AI Bot	Active (green)
Workflow: KTM Mall WhatsApp First Message Handoff	Active (green)
Workflow: KTM Mall Chatbot (Gemini)	Active (green)
Google Sheet	Leads tab created, headers set, Sheet ID: 1HxyzZNhmPXU3LLjk1STczPQFA4XMktWsruQ6MmKXHJc
AI Model	Gemini 2.0 Flash Lite
AI Persona	Sales AI Agent

All 4 workflows are active and pipeline is ready for end-to-end testing.

Last updated: May 2026 | Dreams Media & Marketing